

经网络的语言模型不再通过显性的计算公式对语言符号的概率进行计算，而是利用语料库中的样本对神经网络模型进行训练。本章接下来将分别介绍两类最具代表性的基于神经网络的语言模型：基于 RNN 的语言模型和基于 Transformer 的语言模型。

1.2 基于 RNN 的语言模型

循环神经网络 (Recurrent Neural Network, RNN) 是一类**网络连接中包含环路的神经网络的总称**。给定一个序列，RNN 的环路用于将历史状态叠加到当前状态上。沿着时间维度，历史状态被循环累积，并作为预测未来状态的依据。因此，RNN 可以基于历史规律，对未来进行预测。基于 RNN 的语言模型，以词序列作为输入，基于**被循环编码的上文和当前词**来预测下一个词出现的概率。本节将先对原始 RNN 的基本原理进行介绍，然后讲解如何利用 RNN 构建语言模型。

1.2.1 循环神经网络 RNN

按照推理过程中信号流转的方向，神经网络的正向传播范式可分为两大类：前馈传播范式和循环传播范式。在前馈传播范式中，计算逐层向前，“**不走回头路**”。而在循环传播范式中，某些层的计算结果会通过**环路**被反向引回前面的层中，形成“**螺旋式前进**”的范式。采用前馈传播范式的神经网络可以统称为前馈神经网络 (Feed-forward Neural Network, FNN)，而采用循环传播范式的神经网络被统称为循环神经网络 (Recurrent Neural Network, RNN)。以包含输入层、隐藏层、输出层的神经网络为例。图1.2中展示了最简单的 FNN 和 RNN 的网络结构示意图，可见 FNN 网络结构中仅包含正向通路。而 RNN 的网络结构中除了正向通路，还有一条**环路**将某层的计算结果再次反向连接回前面的层中。



图 1.2: 前馈传播范式与循环传播范式的对比。

设输入序列为 $\{x_1, x_2, x_3, \dots, x_t\}$, 隐状态为 $\{h_1, h_2, h_3, \dots, h_t\}$, 对应输出为 $\{o_1, o_2, o_3, \dots, o_t\}$, 输入层、隐藏层、输出层对应的网络参数分别为 W_I, W_H, W_O 。 $g(\cdot)$ 为激活函数, $f(\cdot)$ 为输出函数。将输入序列一个元素接着一个元素地串行输入时, 对于 FNN, 当前的输出只与当前的输入有关, 即

$$o_t = f(W_O g(W_I x_t)). \quad (1.18)$$

此处为方便对比, 省去了偏置项。独特的环路结构导致 RNN 与 FNN 的推理过程完全不同。RNN 在串行输入的过程中, 前面的元素会被循环编码成隐状态, 并叠加到当前的输入上面。其在 t 时刻的输出如下:

$$\begin{aligned} h_t &= g(W_H h_{t-1} + W_I x_t) = g(W_H g(W_H h_{t-2} + W_I x_{t-1}) + W_I x_t) = \dots \dots \dots \\ o_t &= f(W_O h_t). \end{aligned} \quad (1.19)$$

其中, $t > 0, h_0 = 0$ 。将此过程按照时间维度展开, 可得到 RNN 的推理过程, 如图 1.3 所示。注意, 图中展示的 t 时刻以前的神经元, 都是过往状态留下的“虚影”并不真实存在, 如此展开只是为了解释 RNN 的工作方式。

可以发现, 在这样一个元素一个元素依次串行输入的设定下, RNN 可以将历史状态以隐变量的形式循环叠加到当前状态上, 对历史信息进行考虑, 呈现出螺旋式前进的模式。但是, 缺乏环路的 FNN 仅对当前状态进行考虑, 无法兼顾历史状态。以词序列 {长颈鹿, 脖子, 长} 为例, 在给定“脖子”来预测下一个词是什么的时候, FNN 将仅仅考虑“脖子”来进行预测, 可能预测出的下一词包含“短”, “疼”等等; 而 RNN 将同时考虑“长颈鹿”和“脖子”, 其预测出下一词是“长”的概率将更高, 历史信息“长颈鹿”的引入, 可以有效提升预测性能。

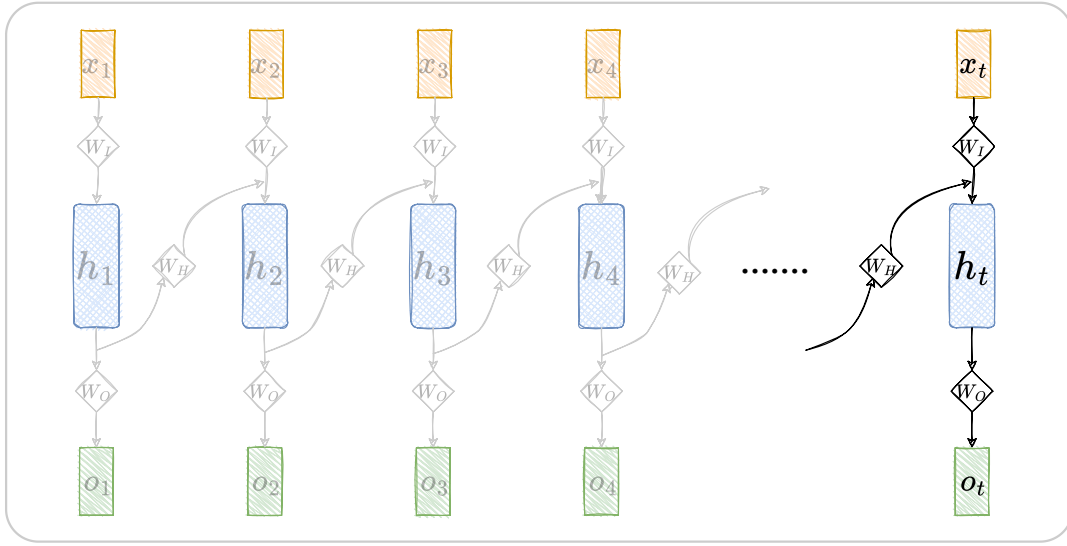


图 1.3: RNN 推理过程从时间维度拆解示意图。

如果 FNN 想要做到对历史信息进行考虑，则需要将所有元素同时输入到模型中去，这将导致模型参数量的激增。虽然，RNN 的结构可以让其在参数量不扩张的情况下实现对历史信息的考虑，但是这样的环路结构给 RNN 的训练带来了挑战。在训练 RNN 时，涉及大量的矩阵联乘操作，容易引发**梯度衰减**或**梯度爆炸**问题。具体分析如下：

设 RNN 语言模型的训练损失为：

$$L = L(x, o, W_I, W_H, W_O) = \sum_{i=1}^t l(o_i, y_i)。 \quad (1.20)$$

其中， $l(\cdot)$ 为损失函数， y_i 为标签。

损失 L 关于参数 W_H 的梯度为：

$$\frac{\partial L}{\partial W_H} = \sum_{i=1}^t \frac{\partial l_t}{\partial o_t} \cdot \frac{\partial o_t}{\partial h_t} \cdot \frac{\partial h_t}{\partial h_i} \cdot \frac{\partial h_i}{\partial W_H}。 \quad (1.21)$$

其中，

$$\frac{\partial h_t}{\partial h_i} = \frac{\partial h_t}{\partial h_{t-1}} \frac{\partial h_{t-1}}{\partial h_{t-2}} \cdots \frac{\partial h_{i+1}}{\partial h_i} = \prod_{k=i+1}^t \frac{\partial h_k}{\partial h_{k-1}}。 \quad (1.22)$$

并且,

$$\frac{\partial h_k}{\partial h_{k-1}} = \frac{\partial g(z_k)}{\partial z_k} W_H. \quad (1.23)$$

其中, $z_k = W_H h_{k-1} + W_I x_k$ 。综上, 有

$$\frac{\partial L}{\partial W_H} = \sum_{i=1}^t \frac{\partial l_t}{\partial o_t} \cdot \frac{\partial o_t}{\partial h_t} \cdot \prod_{k=i}^t \frac{\partial g(z_k)}{\partial z_k} W_H \cdot \frac{\partial h_i}{\partial W_H}. \quad (1.24)$$

从上式中可以看出, 求解 W_H 的梯度时涉及大量的矩阵级联相乘。这会导致其数值被级联放大或缩小。文献 [17] 中指出, 当 W_H 的最大特征值小于 1 时, 会发生梯度消失; 当 W_H 的最大特征值大于 1 时, 会发生梯度爆炸。梯度消失和爆炸导致训练上述 RNN 非常困难。为了解决梯度消失和爆炸问题, GRU [4] 和 LSTM [8] 引入门控结构, 取得了良好效果, 成为主流的 RNN 网络架构。

1.2.2 基于 RNN 的语言模型

对词序列 $\{w_1, w_2, w_3, \dots, w_N\}$, 基于 RNN 的语言模型每次根据当前词 w_i 和循环输入的隐藏状态 h_{i-1} , 来预测下一个词 w_{i+1} 出现的概率, 即

$$P(w_{i+1}|w_{1:i}) = P(w_{i+1}|w_i, h_{i-1}). \quad (1.25)$$

其中, 当 $i = 1$ 时, $P(w_{i+1}|w_i, h_{i-1}) = P(w_2|w_1)$ 。基于此, $\{w_1, w_2, w_3, \dots, w_N\}$ 整体出现的概率为:

$$P(w_{1:N}) = \prod_{i=1}^{N-1} P(w_{i+1}|w_i, h_{i-1}). \quad (1.26)$$

在基于 RNN 的语言模型中, 输出为一个向量, 其中每一维代表着词典中对应词的概率。设词典 D 中共有 $|D|$ 个词 $\{\hat{w}_1, \hat{w}_2, \hat{w}_3, \dots, \hat{w}_{|D|}\}$, 基于 RNN 的语言模型的输出可表示为 $o_i = \{o_i[\hat{w}_d]\}_{d=1}^{|D|}$, 其中, $o_i[\hat{w}_d]$ 表示词典中的词 $\hat{w}_d$ 出现的概率。因此, 对基于 RNN 的语言模型有

$$P(w_{1:N}) = \prod_{i=1}^{N-1} P(w_{i+1}|w_{1:i}) = \prod_{i=1}^N o_i[w_{i+1}]. \quad (1.27)$$

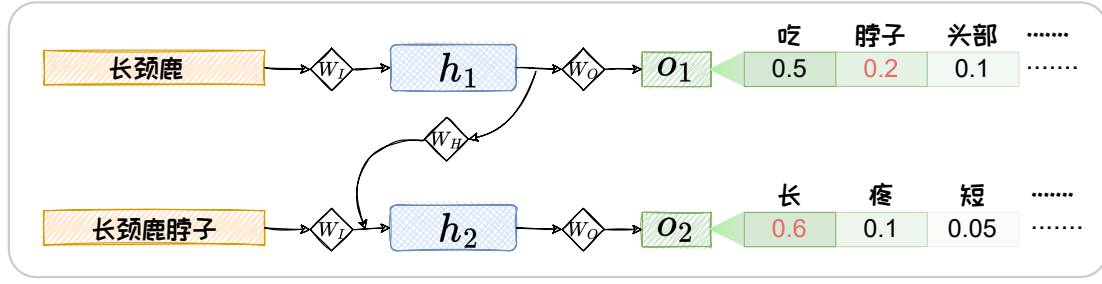


图 1.4: RNN 计算词序列概率示意图。

以下举例对上述过程进行说明。假设词表 $D = \{\text{脖子, 头部, 吃, 长, 疼, 吃, 短}\}$, 基于 RNN 的语言模型计算“长颈鹿脖子长”的概率的过程如图 1.4 所示。

$$P(\text{长颈鹿脖子长}) = P(\text{脖子}|\text{长颈鹿}) \cdot P(\text{长}|\text{脖子}, h_1) = 0.2 \times 0.6 = 0.12. \quad (1.28)$$

基于以上预训练任务，对 RNN 语言模型进行训练时，可选用如下交叉熵函数作为损失函数。

$$l_{CE}(o_i) = - \sum_{d=1}^{|D|} I(\hat{w}_d = w_{i+1}) \log o_i[w_{i+1}] = - \log o_i[w_{i+1}], \quad (1.29)$$

其中， $I(\cdot)$ 为指示函数，当 $\hat{w}_d = w_{i+1}$ 时等于 1，当 $\hat{w}_d \neq w_{i+1}$ 时等于 0。

设训练集为 S ，RNN 语言模型的损失可以构造为：

$$L(S, W_I, W_H, W_O) = \frac{1}{N|S|} \sum_{s=1}^{|S|} \sum_{i=1}^N l_{CE}(o_{i,s}), \quad (1.30)$$

其中， $o_{i,s}$ 为 RNN 语言模型输入第 s 个样本的第 i 个词时的输出。此处为方便表述，假设每个样本的长度都为 N 。在此损失的基础上，构建计算图，进行反向传播，便可对 RNN 语言模型进行训练。上述训练过程结束之后，我们可以直接利用此模型对序列数据进行特征抽取。抽取的特征可以用于解决下游任务。此外，我们还可以对此语言模型的输出进行解码，在“自回归”的范式下完成文本生成任务。在自回归中，第一轮，我们首先将第一个词输入给 RNN 语言模型，经过解码，得到一个输出词。然后，我们将第一轮输出的词与第一轮输入的词拼接，作为第二轮